Combinatorial optimization problems

Outline

Combinatorial optimization problems

Categories of combinatorial optimization problems

Counting the solutions

Combinatorial explosion and brute force

Primal bounds, dual bounds and gap

A glimpse of computational complexity

Combinatorial optimization problems

Introduction: combinatorial optimization

- An optimization problem searches for the best feasible decision.
- Here the decisions are discrete objects: a set of projects, a route through customers, or a subdivision of items into containers.
- We introduce the common language used throughout the course: feasible solution, objective value, instance and optimal value.

Definition 1: combinatorial optimization problem

Given a finite ground set $\mathscr{U} = \{1, \ldots, n\}$, *a family of feasible solutions* $\mathcal{F} \subseteq 2^{\mathscr{U}}$, *and an objective function* $f : \mathcal{F} \to \mathbb{R}$, *find* $\widetilde{S} \in \mathcal{F}$ *such that*

$$f(\widetilde{S}) \geq f(S) \quad \forall S \in \mathcal{F}$$

in maximization form, or $f(\widetilde{S}) \leq f(S)$ *for every* $S \in \mathcal{F}$ *in minimization form.*

Feasible solutions, objectives and instances

- $\mathcal{F}$ is finite because $|2^{\mathcal{U}}| = 2^n$; therefore an optimum exists whenever $\mathcal{F} \neq \emptyset$.
- Often f is additive: each element j has weight c_j and

$$f(S) = \sum_{j \in S} c_j.$$

- A **problem** has symbolic data. An **instance** I fixes the data; $z(I)$ denotes its optimal value.
- In the problems of this course, $\mathcal{F}$ is described by rules and is too large to list.

Example

Let $\mathscr{U} = \{1, 2, 3, 4\}$, $\boldsymbol{c} = (7, 6, 4, 3)$, and maximize $f(S) = \sum_{j \in S} c_j$ over

$$\mathcal{F} = \{\emptyset, \{1\}, \{2\}, \{3\}, \{4\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}, \{3, 4\}, \{2, 3, 4\}\}.$$

- $f(\{1, 3\}) = 11$ and $f(\{2, 3, 4\}) = 13$.
- $\{1, 2\}$ is not feasible.

Summary

- Combinatorial optimization studies optimization over discrete structures: graphs, subsets, permutations, sequences, partitions, coverings and packings.
- The feasible region is finite, but it is usually enormous.
- Specialized algorithms explore the solution space without enumerating it in full.

Categories of combinatorial optimization problems

Introduction: categories

- A decision category identifies the combinatorial object representing a solution.
- It also suggests natural decision variables and determines how many candidate solutions must be considered.
- The same application may combine more than one category.

Categories of decisions

Category	Object	Natural variables
IN/OUT	subset	$x_j \in \{0, 1\}$
counting	integer quantities	$x_j \in \mathbb{Z}_{\geq 0}$
assignment	function to labelled resources	$x_{ij} \in \{0, 1\}$
ordering	permutation or cyclic ordering	successor variables x_{ij}
configuration	partition into interchangeable groups	group variables x_{ij}

- Assignments have distinct resources; partitions have interchangeable groups.

Application areas

- production planning and lot sizing;
- scheduling and maintenance;
- facility location;
- packing and cutting;
- logistics and transportation.
- personnel management;
- network design;
- finance and telecommunications;
- energy and healthcare;
- crop, sport and patrol planning.

The parcels we receive, the electricity we use and the flights we take are planned by solving combinatorial optimization problems.

The problems of the course

- **Knapsack**: select valuable projects under a budget.
- **Bin packing**: load items into the fewest identical containers.
- **Vertex coloring**: assign the fewest exam days to conflicting exams.
- **Clique**: select pairwise compatible vertices.
- **Travelling salesman**: find the shortest round trip.
- **Scheduling**: distribute jobs to complete them early.
- **Lot sizing**: choose production periods and quantities.

Summary

- Categories link the decision, the combinatorial object and the natural variables.
- IN/OUT and counting decisions occur in knapsack and lot sizing; partitioning occurs in bin packing and coloring; ordering occurs in routing.
- This classification guides both modelling and algorithms.

Counting the solutions

Introduction: counting

- Before solving a problem, count its candidate solutions.
- The count depends on the combinatorial object: subsets, orderings, cyclic orderings or partitions.
- Candidate solutions need not be feasible; in the small instance, only 11 of the 16 subsets are feasible.

Proposition 1

A set with n elements has 2^n subsets.

A subset is identified by $\boldsymbol{x} \in \{0,1\}^n$, where $x_j = 1$ exactly when $j \in S$. Each component has two independent choices, hence $2 \cdot 2 \cdots 2 = 2^n$ vectors. Equivalently,

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n. \quad \square$$

Proposition 2

For $n \geq 3$, the number of cyclic orderings, identifying rotations and reversals, is

$$\frac{(n-1)!}{2}.$$

Fix the starting element. The other $n-1$ elements can be ordered in $(n-1)!$ ways; each cycle is counted twice, once in each direction. For $n = 4$, the three cycles are $(1, 2, 3, 4)$, $(1, 2, 4, 3)$ and $(1, 3, 2, 4)$.

Partitions and Bell numbers

- A partition is a collection of non-empty, pairwise disjoint subsets whose union is the ground set.
- The number of partitions of n elements is the Bell number:

$$B(0) = 1, \qquad B(n) = \sum_{k=0}^{n-1} \binom{n-1}{k} B(k).$$

- The first values are $1, 1, 2, 5, 15, 52, 203, 877, \ldots$.
- Assignments of n elements to m labels are m^n; identical labels instead give partitions into at most m blocks.

Example

object for $n = 5$	number
subsets	$2^5 = 32$
orderings	$5! = 120$
cyclic orderings	$4!/2 = 12$
partitions	$B(5) = 52$
assignments to 3 labels	$3^5 = 243$
partitions into at most 3 blocks	41

Summary

- IN/OUT decisions have 2^n candidates; orderings have $n!$; round trips have $(n-1)!/2$; partitions have $B(n)$.
- Interchangeable labels must not be counted several times.
- These counts anticipate the difficulty of exhaustive search.

Combinatorial explosion and brute force

Introduction: brute force

- A finite feasible region can always be solved by complete enumeration.
- Brute force enumerates candidates, discards infeasible ones and retains the best feasible solution.
- The method is exact, but the number of candidates grows exponentially or faster.

Brute force

1. Set $\overline{S} \leftarrow$ none and $\mathrm{LB} \leftarrow -\infty$.
2. For every candidate S:
   - discard S if it is infeasible;
   - if S is feasible and $f(S) > \mathrm{LB}$, set $\overline{S} \leftarrow S$ and $\mathrm{LB} \leftarrow f(S)$.
3. Return $\widetilde{S} = \overline{S}$ and $z(I) = \mathrm{LB}$.

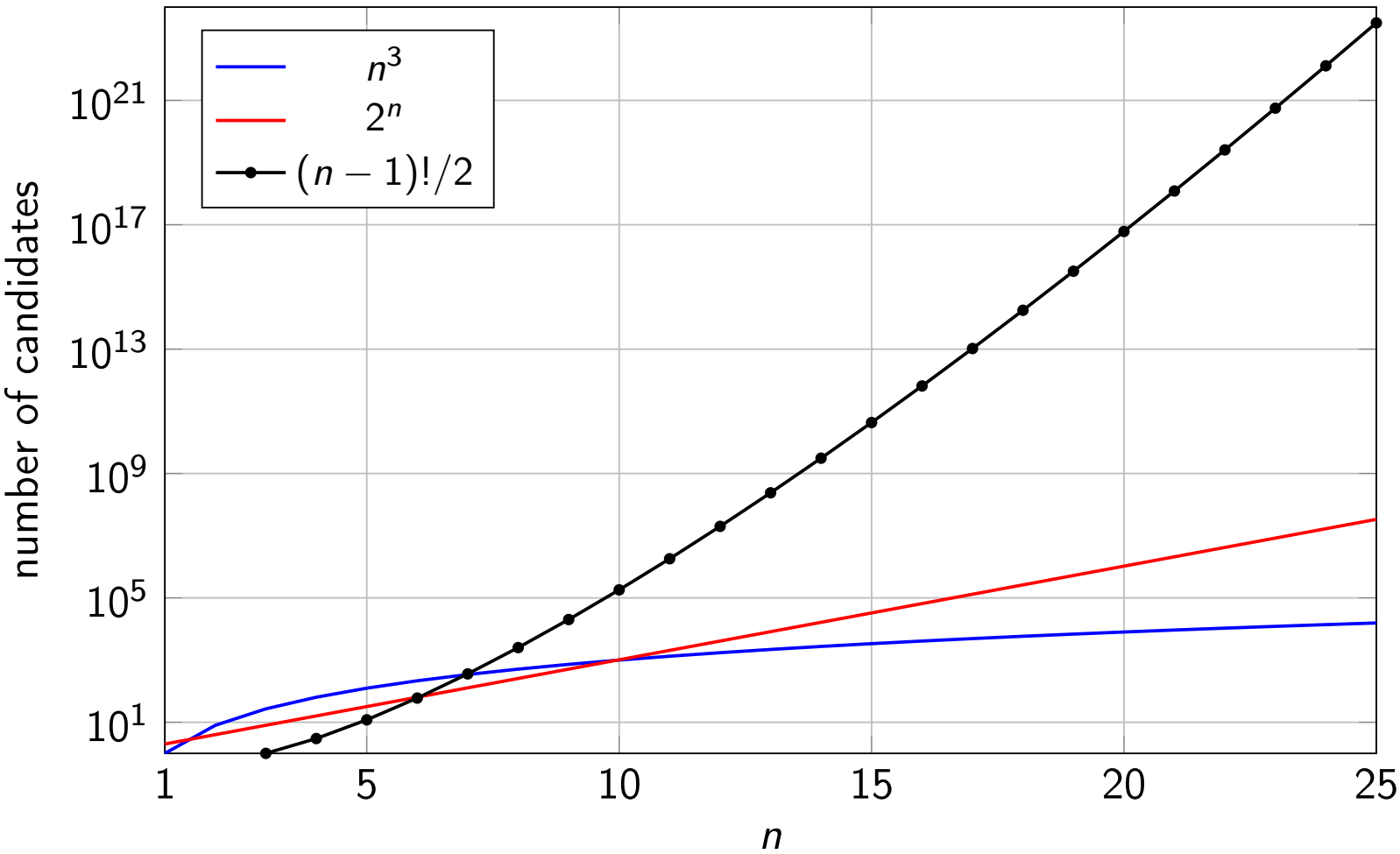
Example

For the small instance, brute force examines all $2^4 = 16$ subsets. Eleven are feasible; the best is

$$\widetilde{S} = \{2, 3, 4\}, \qquad z(I) = 13.$$

The subset $\{1, 2\}$ has the same value but is infeasible. Enumeration proves optimality only after all candidates have been examined.

Exponential growth



Computing times

At 10^9 candidates per second:

n	2^n	time	cyclic orderings for $n = 20$
40	$1.10 \cdot 10^{12}$	18.3 minutes	1.93 years
50	$1.13 \cdot 10^{15}$	13.0 days	
60	$1.15 \cdot 10^{18}$	36.5 years	
80	$1.21 \cdot 10^{24}$	$3.83 \cdot 10^7$ years	

A computer 1000 times faster gains only about ten elements for subsets, and fewer than three from $n = 15$ for cyclic orderings.

Summary

- Complete enumeration is possible only for very small instances.
- A constant speed-up does not defeat exponential growth.
- Useful algorithms exclude whole regions of the solution space without enumerating them.

Primal bounds, dual bounds and gap

Introduction: bounds

- Instead of computing the optimum directly, enclose it between two numbers.
- A feasible solution supplies one side; an argument valid for all feasible solutions supplies the other.
- When the two bounds coincide, they certify optimality.

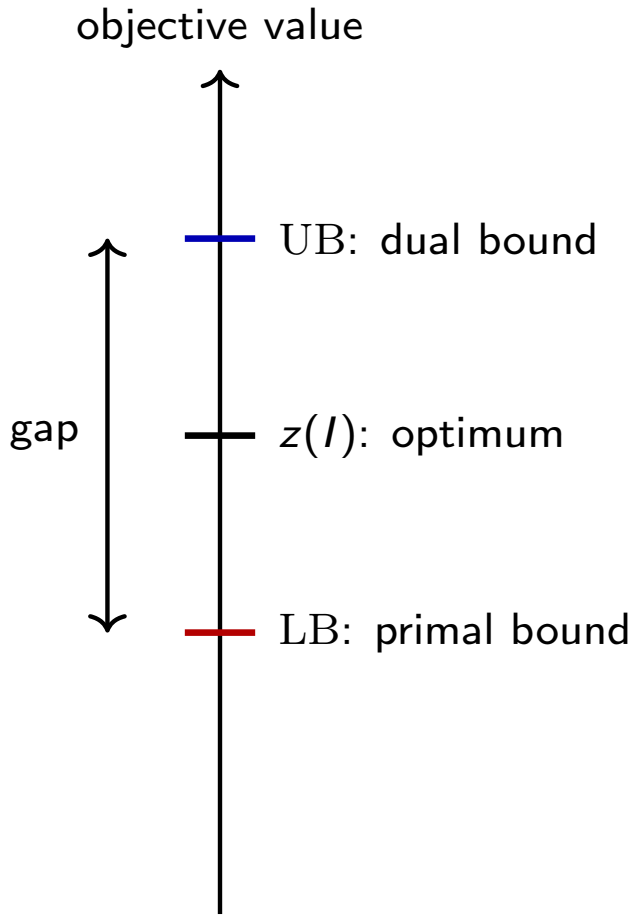
Definition 2: primal and dual bounds

*For an instance I, a **primal bound** is $f(\overline{S})$ for a feasible solution $\overline{S}$. A **dual bound** is β such that $f(S) \leq \beta$ for every feasible S in maximization (or $f(S) \geq \beta$ in minimization).*

$$\text{maximization:} \qquad \mathrm{LB} = f(\overline{S}) \leq z(I) \leq \mathrm{UB}.$$

In minimization, the primal bound is the upper bound and the dual bound is the lower bound.

Bounds in maximization



Primal bounds come from feasible solutions, often from heuristics.
Dual bounds come from relaxations or combinatorial arguments.

Definition 3: gap

If $\mathrm{LB} \leq z(I) \leq \mathrm{UB}$, *the absolute gap is* $\mathrm{UB} - \mathrm{LB}$ *and, for* $\mathrm{LB} \neq 0$, *the relative gap is*

$$gap(I) = \frac{\mathrm{UB} - \mathrm{LB}}{|\mathrm{LB}|}.$$

- The denominator is the lower bound.
- A zero gap certifies that the feasible solution is optimal.
- If all objective values are integer, $\lfloor \beta \rfloor$ is a dual bound in maximization and $\lceil \beta \rceil$ in minimization.

Example

For the small maximization instance:

- $\{1, 3\}$ gives $\mathrm{LB} = 11$.
- Positivity gives $\mathrm{UB} = 20$; the incompatibility of 1 and 2 improves it to $\mathrm{UB} = 14$.
- $\{2, 3, 4\}$ gives $\mathrm{LB} = 13$, so the relative gap is $1/13 \simeq 7.7\%$.
- Only $\{2, 3, 4\}$ is feasible with three elements, while any set with at most two elements has value at most $7 + 6 = 13$.

Thus $\mathrm{LB} = \mathrm{UB} = 13$ and $\{2, 3, 4\}$ is optimal without listing all subsets.

Summary

- In maximization, primal means lower and dual means upper; in minimization the roles are reversed.
- A gap measures how far the current certificate may be from optimality.
- Exact methods close the gap by improving primal or dual bounds.

A glimpse of computational complexity

Introduction: complexity

- Complexity asks whether a problem can be solved in time polynomial in the number of bits of its input.
- The decision version asks a yes/no question, such as whether a feasible solution has value at least k.
- These notions explain why brute force is inadequate and why several algorithmic approaches coexist.

Definition 4: P, NP and NP-hardness

- ***P***: *decision problems solved by a polynomial algorithm.*
- ***NP***: *decision problems whose yes instances have certificates checked in polynomial time.*
- ***NP-complete***: *problems in NP to which every NP problem reduces in polynomial time.*
- *An optimization problem is* ***NP-hard*** *when its decision version is NP-complete.*

Definition 5: pseudo-polynomial algorithms

*An algorithm polynomial in the instance size and in the largest numerical value M is **pseudo-polynomial**. It is not polynomial in the input length, since writing M requires only about $\log_2 M$ bits.*

- A strongly NP-hard problem remains NP-hard when its numbers are polynomially bounded.
- A weakly NP-hard problem can admit a pseudo-polynomial algorithm.

Typical mistakes

- Do not confuse candidate solutions with feasible solutions.
- Do not exchange primal and dual bounds.
- Round a dual bound only when every objective value is integer.
- Do not count rotations, reversals or renamings of interchangeable labels as distinct solutions.
- NP-hard means difficult in the worst case, not unsolvable.

Summary

- Combinatorial optimization combines finite solution spaces with very fast growth.
- Candidate counts, bounds and the gap give the basic language for assessing a method.
- The course studies exact methods, heuristics and approximation algorithms for NP-hard problems.